VPC Peering Connection

Technical Project Report

Project platform: Amazon Web Services (AWS)

Architecture basis: user-provided VPC Peering Connection diagram

1. Abstract

This report describes an AWS network architecture intended to connect two Virtual Private Clouds through VPC peering. The supplied diagram includes EC2 instances, public and private subnet labels, route tables, an Internet Gateway, and a NAT Gateway. The report explains the purpose of each component and outlines the configuration and validation steps needed to turn the diagram into a working deployment.

2. Architecture Overview

The diagram places two VPCs inside one AWS Region labelled Ohio. VPC 1 contains a public subnet with an EC2 instance and a private subnet with another EC2 instance. A NAT Gateway is shown between the public and private workload path. VPC 2 contains an EC2 instance in a subnet labelled public. Gateway symbols and route tables are also shown. The diagram is conceptual; the exact deployed configuration must be checked in the AWS Console.

3. Main Components

- **Amazon VPC:** A logically isolated virtual network in AWS. Each VPC has an IPv4 CIDR range.
- **Subnets:** Smaller IP ranges within a VPC. A subnet is public or private based on its route configuration and the addressing/configuration of its resources.
- **EC2 instances:** Virtual servers that host workloads and can be used to test connectivity.
- **VPC peering connection:** A private network connection between two VPCs. Peering does not automatically create routes; route tables must be configured.
- **Route tables:** Define where traffic is directed. Each VPC needs a route to the other VPC's CIDR via the peering connection for bidirectional communication.
- **Internet Gateway:** Provides a route target for internet connectivity for eligible resources in a VPC; it does not by itself make every subnet or instance public.
- **NAT Gateway:** Allows instances in a private subnet to initiate supported outbound connections to external destinations when routes and addressing are configured. It is not required for VPC-to-VPC peering.

4. Addressing Information Shown in the Diagram

The diagram visibly labels VPC 1 as 10.0.0.0/16 and shows 192.168.0.0/24 near VPC 2. Several route labels appear to contain 172.16.0.0, 172.16.1.0, and 172.16.2.0. These labels are not sufficiently clear to confirm the complete VPC and subnet address plan.

Before deployment, define a clear, non-overlapping CIDR plan for both VPCs and all subnets. The route destinations must match the actual remote VPC CIDR. Do not copy the ambiguous orange route labels without verifying them.

5. Proposed Implementation Procedure

1. Choose one AWS Region and confirm the account has permission to manage VPC, EC2, route tables, gateways, security groups, and peering connections.
2. Create VPC 1 and VPC 2 with non-overlapping IPv4 CIDR ranges.
3. Create the required subnets in each VPC and associate them with the intended route tables.
4. Attach an Internet Gateway to the VPC that needs public internet access. Configure a default route only in the appropriate public route table.
5. If private instances require outbound internet access, configure a NAT Gateway in a public subnet and route private-subnet internet-bound traffic through it. This is optional and may incur charges.
6. Launch the EC2 instances in their intended subnets. Avoid public IPv4 addresses on instances intended to remain private.
7. Create a VPC peering connection between the two VPCs and accept the request if required.
8. Add a route for the remote VPC CIDR to the peering connection in the relevant route tables on both sides.
9. Configure security groups to allow only the required application, SSH, or ICMP traffic from approved sources.
10. Test connectivity using the intended private IP addresses and save actual results and screenshots.

6. Routing and Security Considerations

- VPC CIDR ranges must not overlap for ordinary VPC peering.
- The peering connection must be active, and both sides need appropriate routes.
- Security groups must permit the required inbound traffic; network ACLs may also affect traffic.
- A VPC peering connection is non-transitive. It does not automatically route traffic through a third VPC.
- VPC peering does not provide VPC 2 with internet access through VPC 1's Internet Gateway.
- Limit SSH access to a trusted source. Do not expose SSH broadly to 0.0.0.0/0.
- NAT Gateway and public IPv4 usage may incur charges. Delete lab resources when finished.

7. Testing and Validation Plan

11. **Architecture:** Confirm both VPCs and subnets exist and that the CIDRs do not overlap.
12. **Peering:** Confirm the peering connection status is Active and the requester/accepter VPCs are correct.
13. **Routes:** Confirm each relevant route table has a route to the remote VPC CIDR through the peering connection.
14. **Security:** Review security group rules for the intended source addresses and ports.
15. **Connectivity:** From an authorized test host, test the destination's private IP using the selected protocol and record the real output.
16. **Negative test:** Temporarily remove a specific test permission in a controlled lab, verify that the test fails, then restore the rule and verify recovery.

No test results are asserted in this report because the supplied image is an architecture diagram, not evidence of a completed deployment. Record observed results only after running the tests.

8. Potential Issues and Troubleshooting

- **Ping fails:** Check the active peering status, both route tables, destination private IP, security group ICMP rules, and network ACLs.

- **SSH fails:** Check the key pair, username, TCP port 22, allowed source, instance state, and route in both directions.
- **Peering cannot be created:** Check whether the VPC CIDR ranges overlap.
- **Private instance cannot reach the internet:** Check whether internet egress is required and whether NAT Gateway routes are configured. This issue is separate from peering connectivity.
- **Traffic works in only one direction:** Check the return route and the destination security rules.

9. Cost and Cleanup

Review AWS pricing and available credits before deployment. EC2, EBS, NAT Gateway, and public IPv4 resources can incur charges. After capturing evidence, terminate lab EC2 instances and remove the peering connection, routes, security groups, subnets, custom route tables, gateways, and VPCs created for the project. Check billing for remaining chargeable resources.

10. Conclusion

The architecture illustrates the key AWS networking components used when connecting separate VPCs. Successful private communication depends on a valid peering connection, non-overlapping address ranges, correct routes on both sides, and security rules that permit the required traffic. The next step is to validate the CIDR labels in the supplied diagram and implement the design in the AWS Console, then include screenshots and real test results.

Appendix: Information to Complete During Deployment

AWS Region: ap-south-1 (Mumbai; example Region specified in the source PDF)

VPC 1 ID and CIDR: Actual VPC ID to record after creation; ewb-vpc-a — 10.10.0.0/16

VPC 2 ID and CIDR: Actual VPC ID to record after creation; ewb-vpc-b — 10.20.0.0/16

Peering connection ID/status: Actual pcx- ID to record; expected status after acceptance: Active

EC2 instance IDs and private IPs: Record actual instance IDs; ewb-bastion — 10.10.1.10 + public IPv4; ewb-app-a — 10.10.2.10 (no public IP); ewb-app-b — 10.20.1.10 (no public IP)

Route table IDs and route targets: Record actual IDs; ewb-rt-a-public: 0.0.0.0/0 → ewb-igw-a and 10.20.0.0/16 → pcx-ID; ewb-rt-a-private: 10.20.0.0/16 → pcx-ID; ewb-rt-b-private: 10.10.0.0/16 → pcx-ID

Connectivity test results: Expected criteria from source PDF—before peering, App-A ping to 10.20.1.10 gets no replies; after routes/security rules, forward and reverse ping should return 4 replies with 0% packet loss; SSH to App-B through bastion should work. Replace with actual observed results.

Cleanup verification date: Complete after deleting lab resources; enter the actual date of verification.